

robbit-esp Development Manual

Kise Laboratory
Created: September 2025

robbit-esp: Overview

- robbit-esp is an accessible two-wheeled self-balancing robot controlled by an ESP32-C3.

- Development Workflow

Assemble robbit-esp



Upload the Program

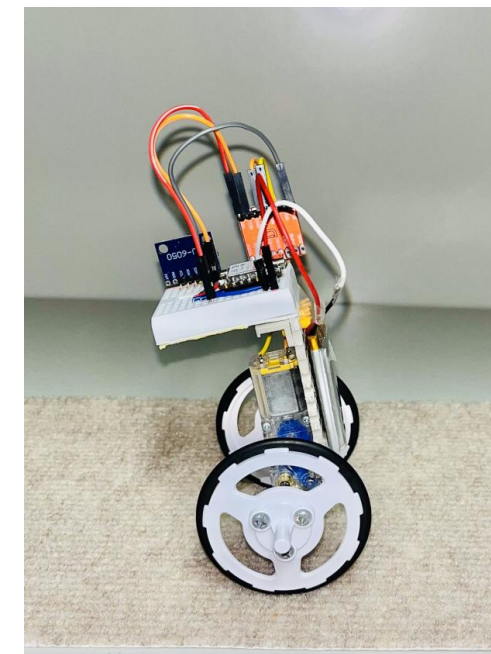
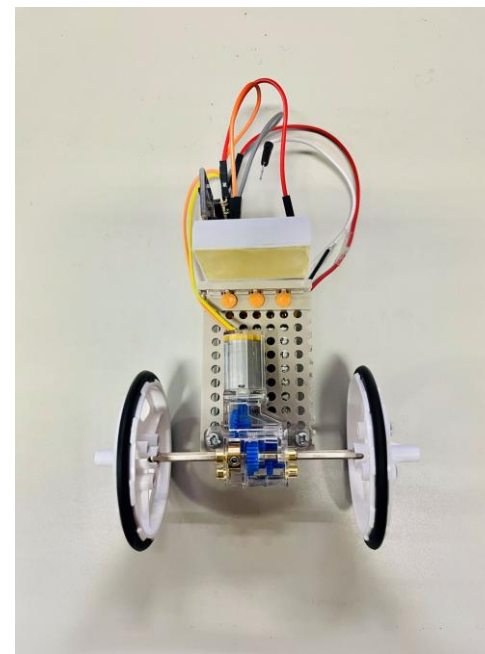


Operation Check



Parameter Tuning

robbit-esp



Assembling robbit-esp: Purchase Components

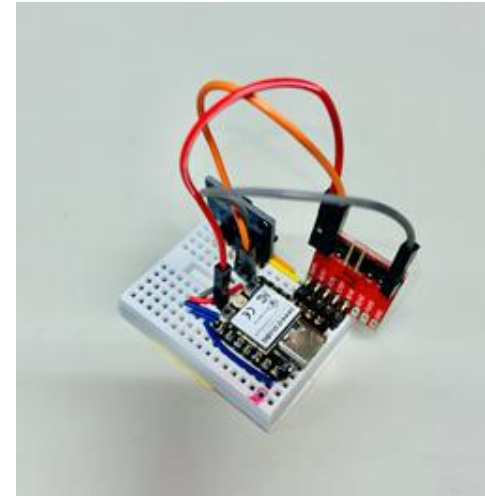
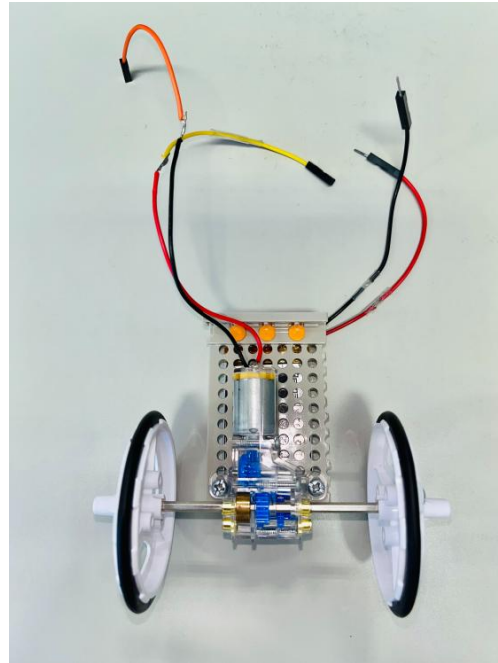
- Assemble robbit-esp using the following steps.

Purchase
Components

Build the
Chassis

Build the
Control Unit

Assembly



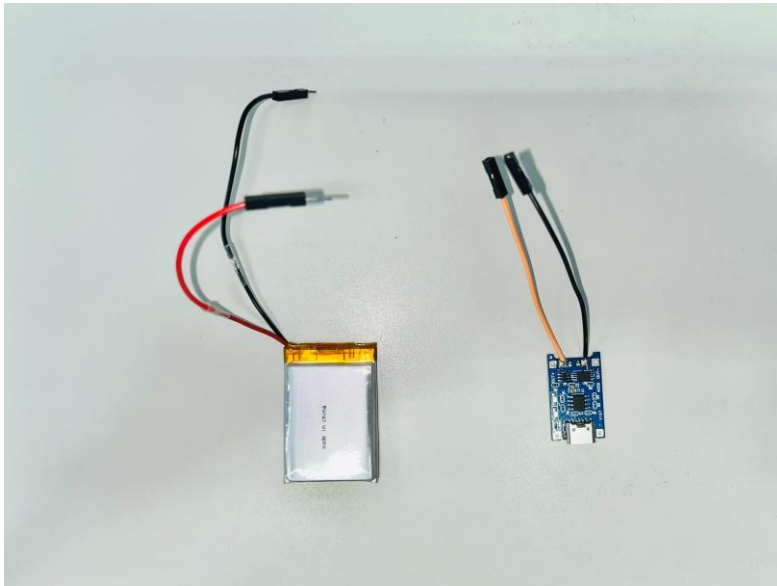
Assembling robbit-esp: Purchase Components

■ Purchase the components listed below.

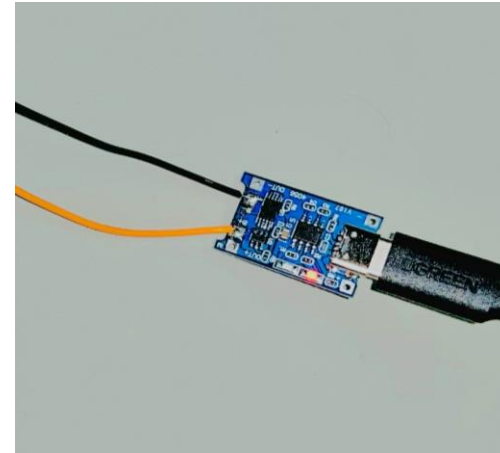
Component	Product	Quantity
Microcontroller	XIAO ESP32-C3	1
Sensor	MPU-6050	1
Motor	Mini Motor Standard Gearbox 70188	1
Tires	Slim Tire Set (55mm Dia.) 70193	1
Motor Driver	TB6612FNG	1
Battery	EEMB Lithium-Ion Battery 653042	1
Plate	Universal Plate Set 70157	1

Assembling robbit-esp: Build the Chassis

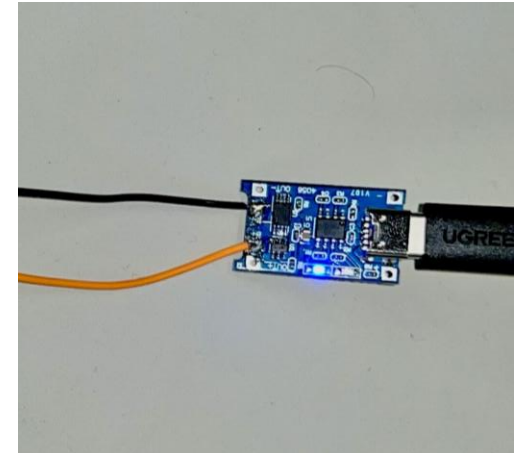
- Solder the battery and charging module as shown.
- After soldering, connect the battery to the charging module.
- The charging-module LED indicates the battery status.
(Red: charging; blue: fully charged)



Soldering Example



Red: Charging



Blue: Fully Charged

Charging-Module LED

Assembling robbit-esp: Build the Chassis

- Cut the universal plate to size.
- Size the universal plate to match the control unit that will be built later.
- Leave an area for attaching the control unit, as shown below.



Universal Plate After Cutting



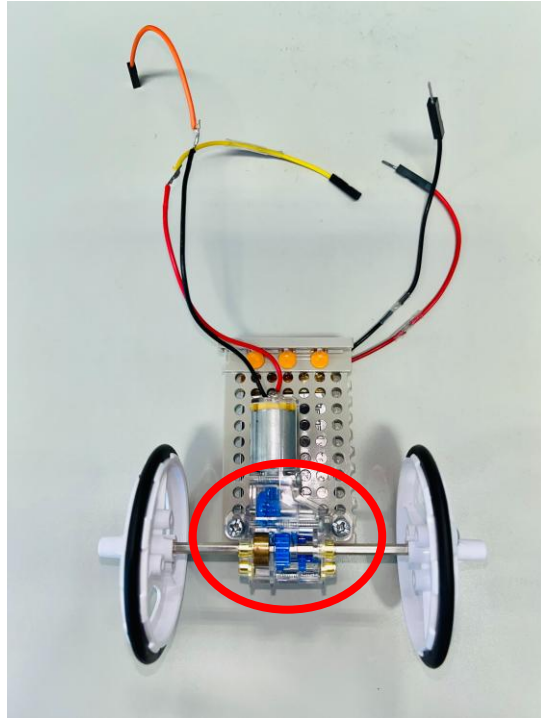
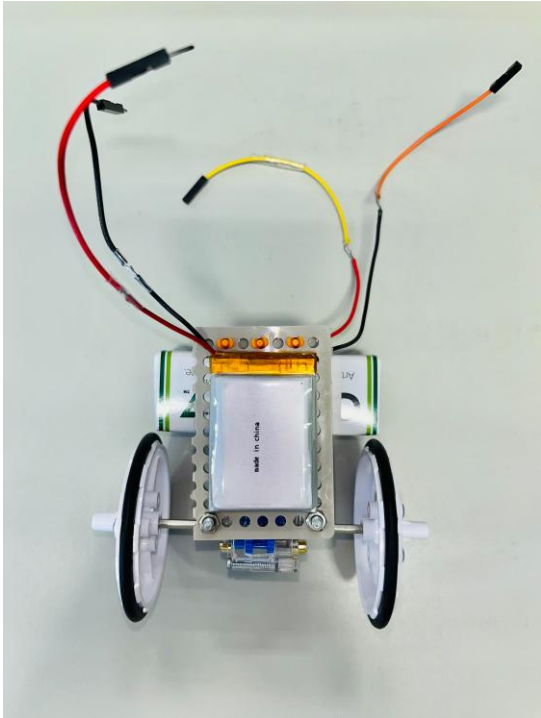
Control-Unit Attachment Area

Assembling robbit-esp: Build the Chassis

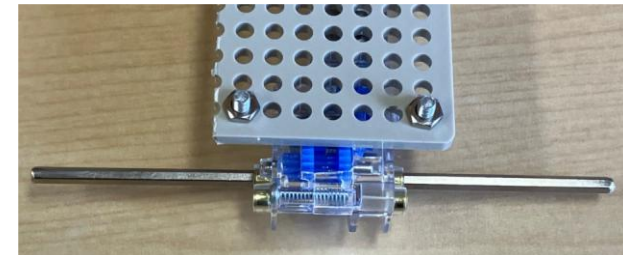
- Assemble the TAMIYA Mini Motor Standard Gearbox 70188 and Slim Tire Set (55 mm Dia.) 70193.
- The gearbox assembly instructions are shown on the next page. Required components and steps are highlighted in yellow.
- We recommend starting with the 55 mm tires. Smaller tires are also included, but they may make control more difficult.

Assembling robbit-esp: Build the Chassis

- Attach the tires after assembling the motor.
- Finally, attach the battery and motor to the universal plate to complete the chassis.



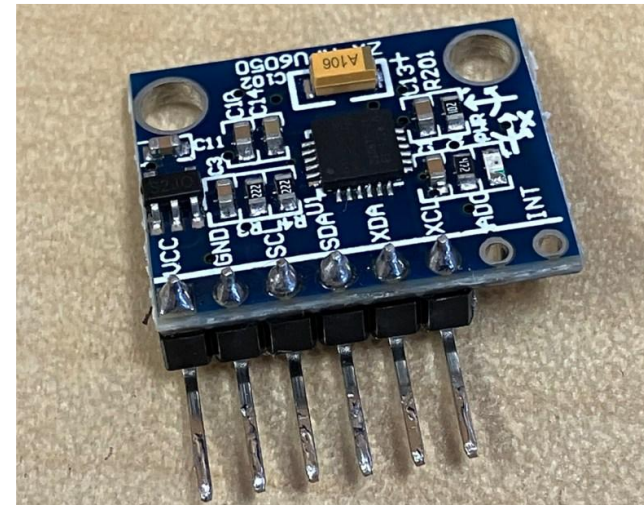
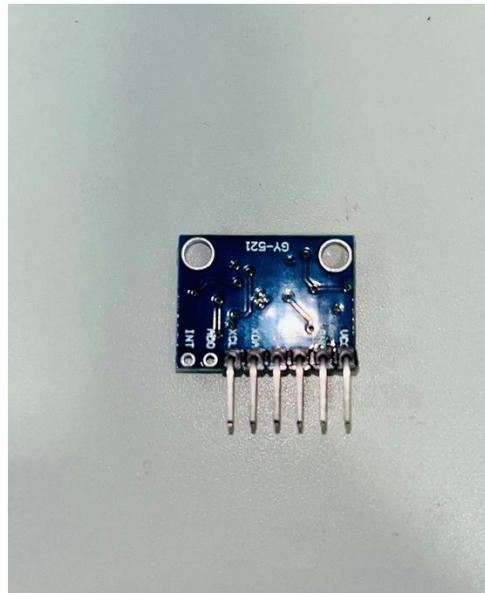
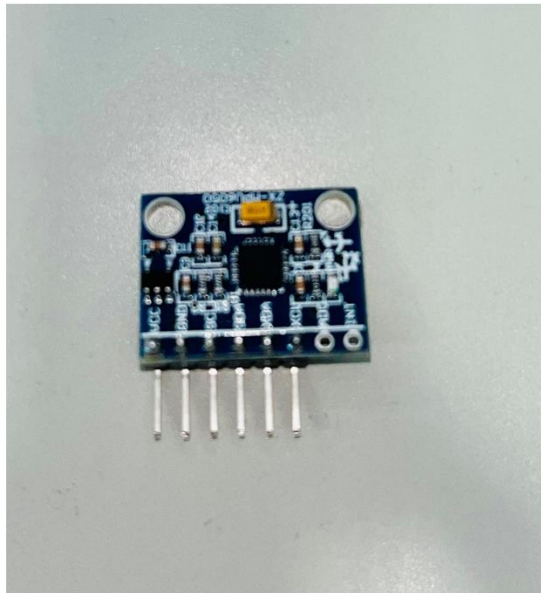
Fasten Securely with Screws



Completed Chassis

Assembling robbit-esp: Build the Control Unit

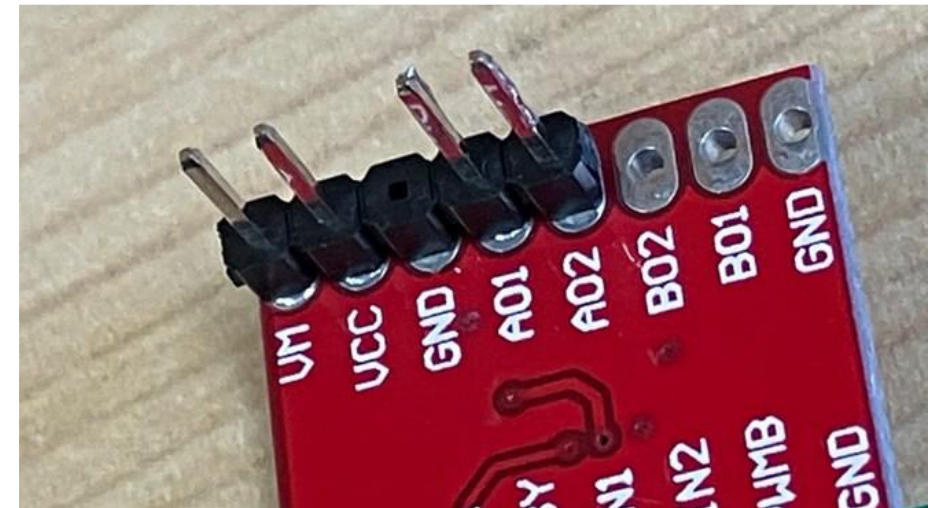
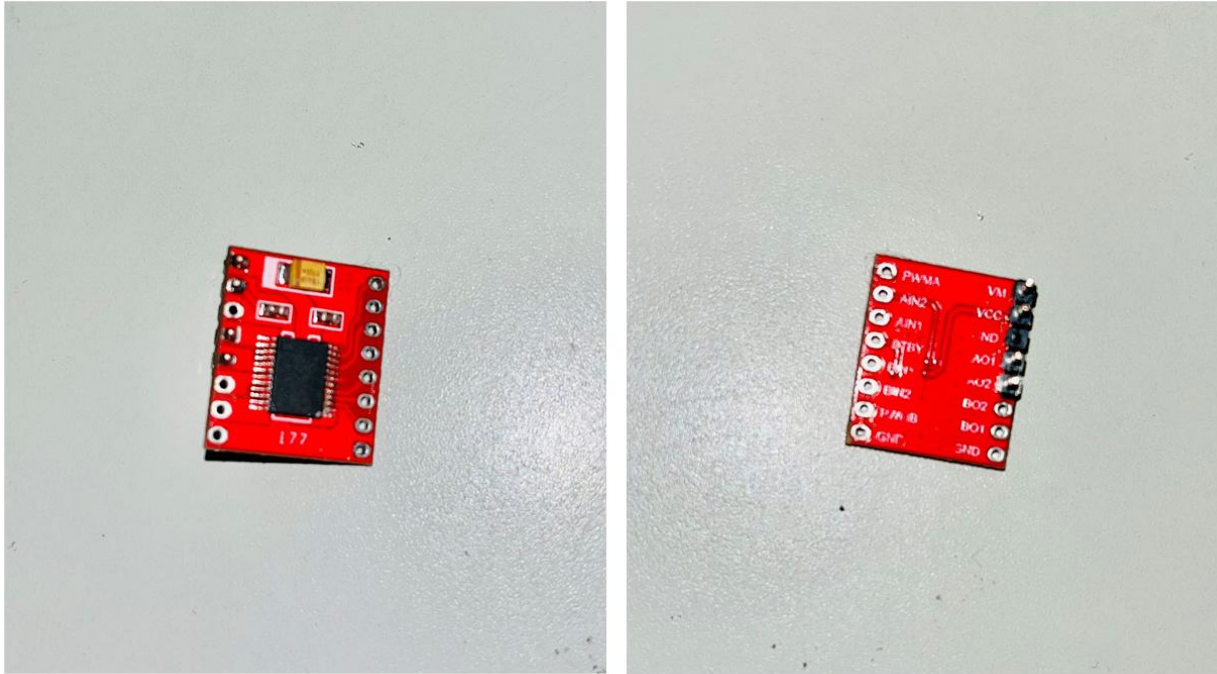
- Solder pin headers to the MPU-6050 sensor. Although the IMU operates with VCC, GND, SCL, and SDA connected, soldering headers to every connection point is recommended for better mechanical stability.



Soldering Example

Assembling robbit-esp: Build the Control Unit

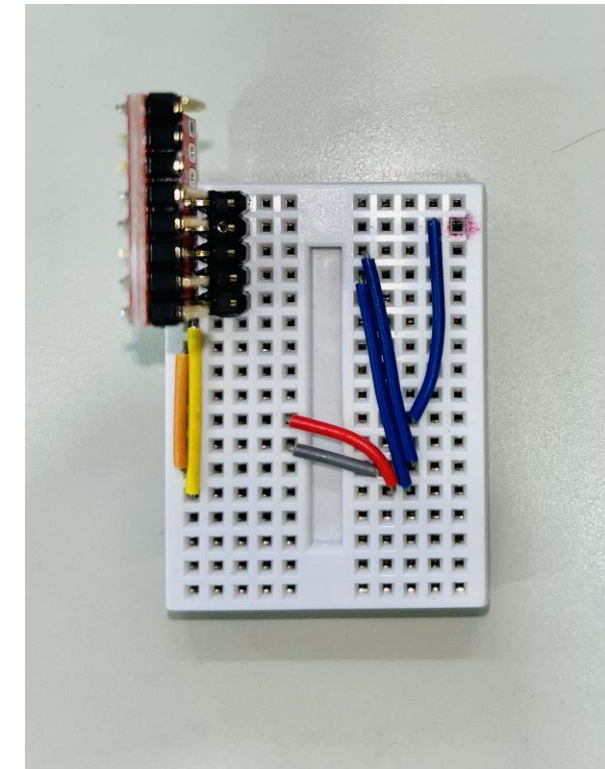
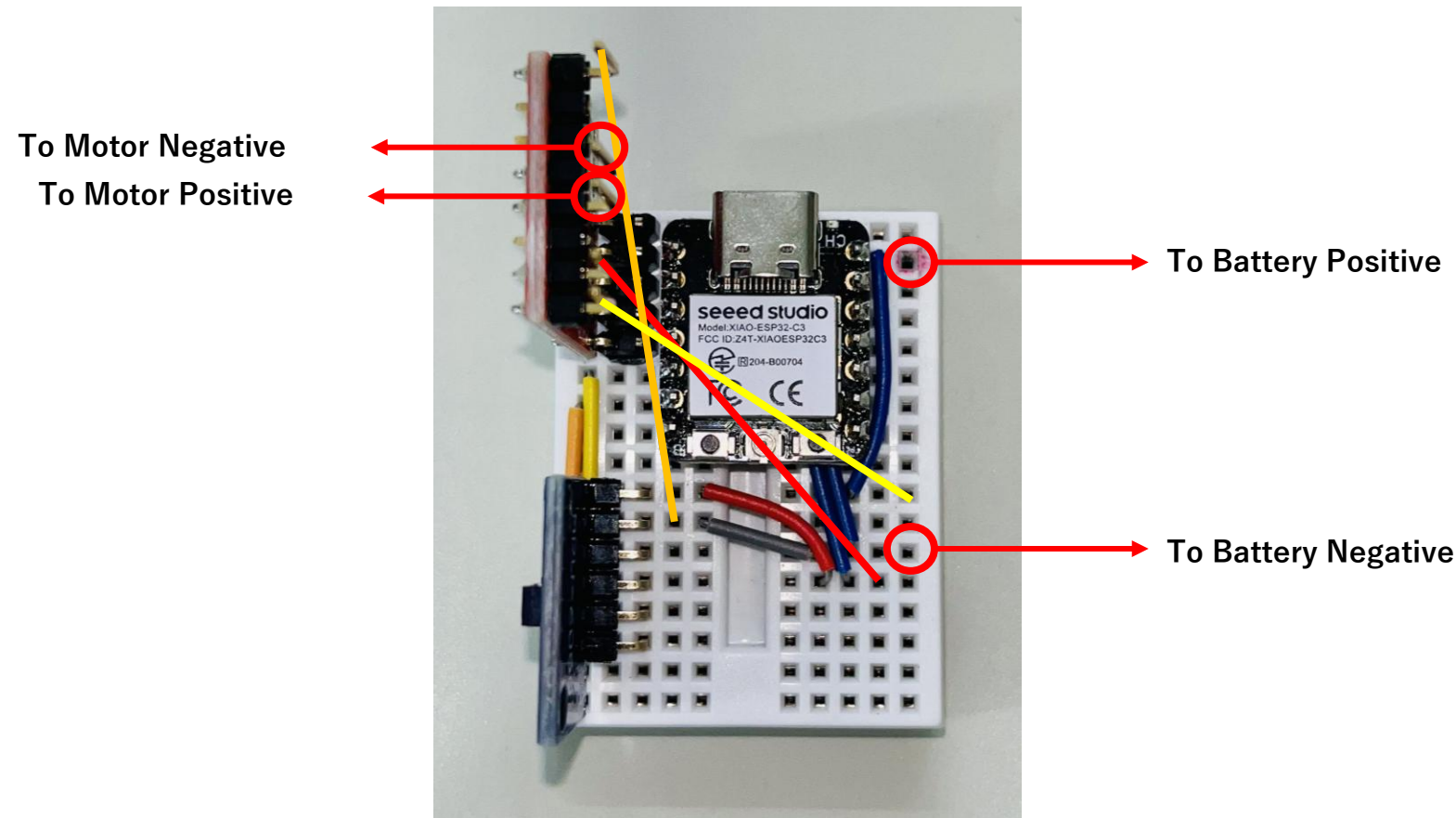
- Solder headers or wires to VM, VCC, GND, A01, and A02 on the TB6612FNG motor driver.
- Remove the unused GND pin.



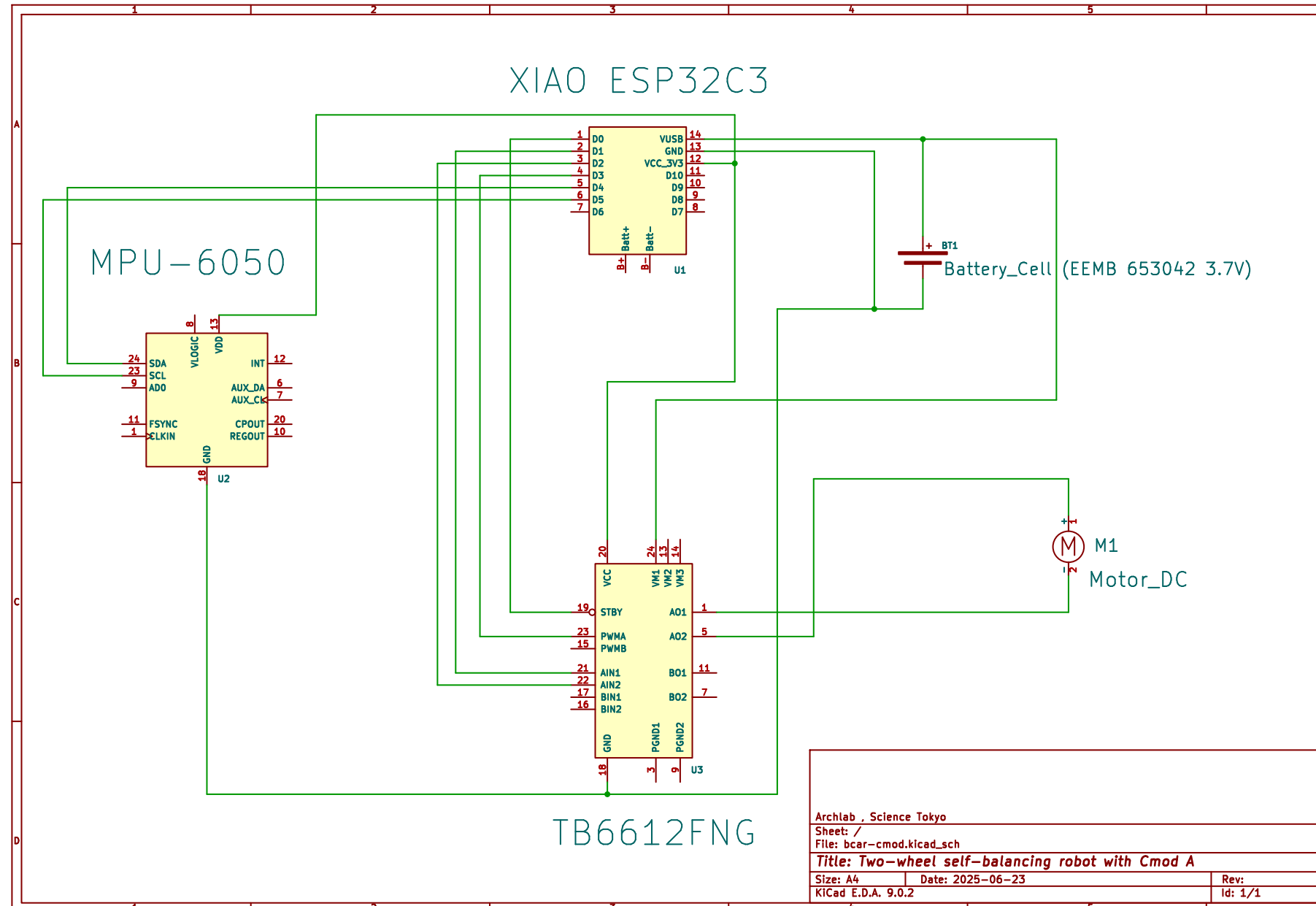
Soldering Example

Assembling robbit-esp: Build the Control Unit

- Prepare a breadboard or universal PCB.
- Wire and solder the prepared board according to the circuit diagram.

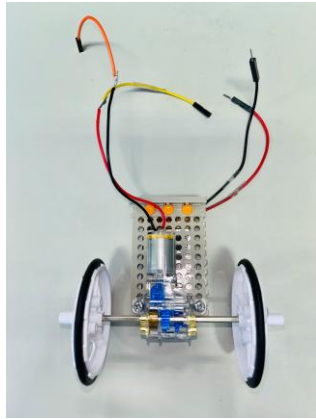


Assembling robbit-esp: Build the Control Unit

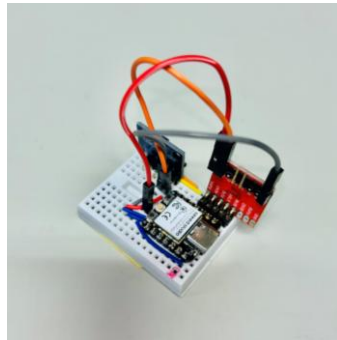


Assembling robbit-esp: Build the Control Unit

- Attach the completed chassis and control unit with double-sided tape to complete robbit-esp.

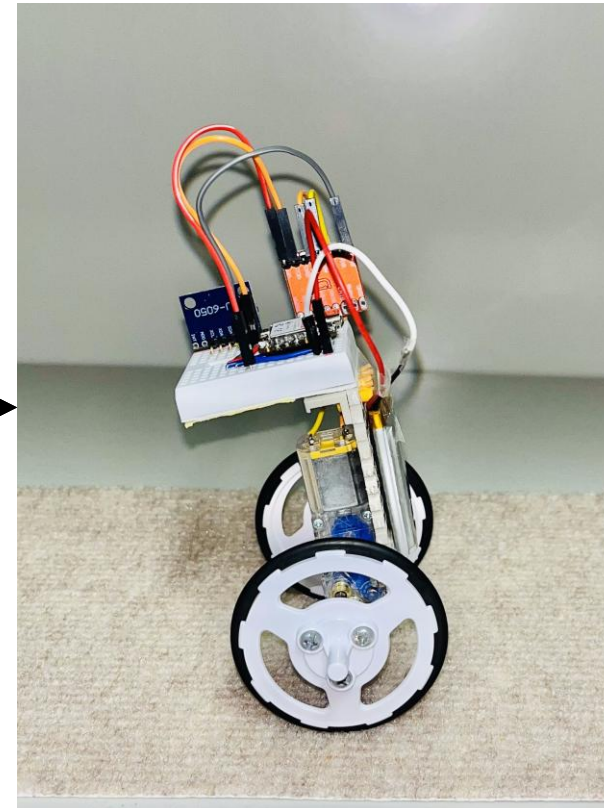


Chassis



Control Unit

Assembly



robbit-esp

Program Upload: Environment Setup

- Use the Arduino IDE to upload the program to the completed robbit-esp. Download it if it is not already installed.



Arduino IDE 2.3.6

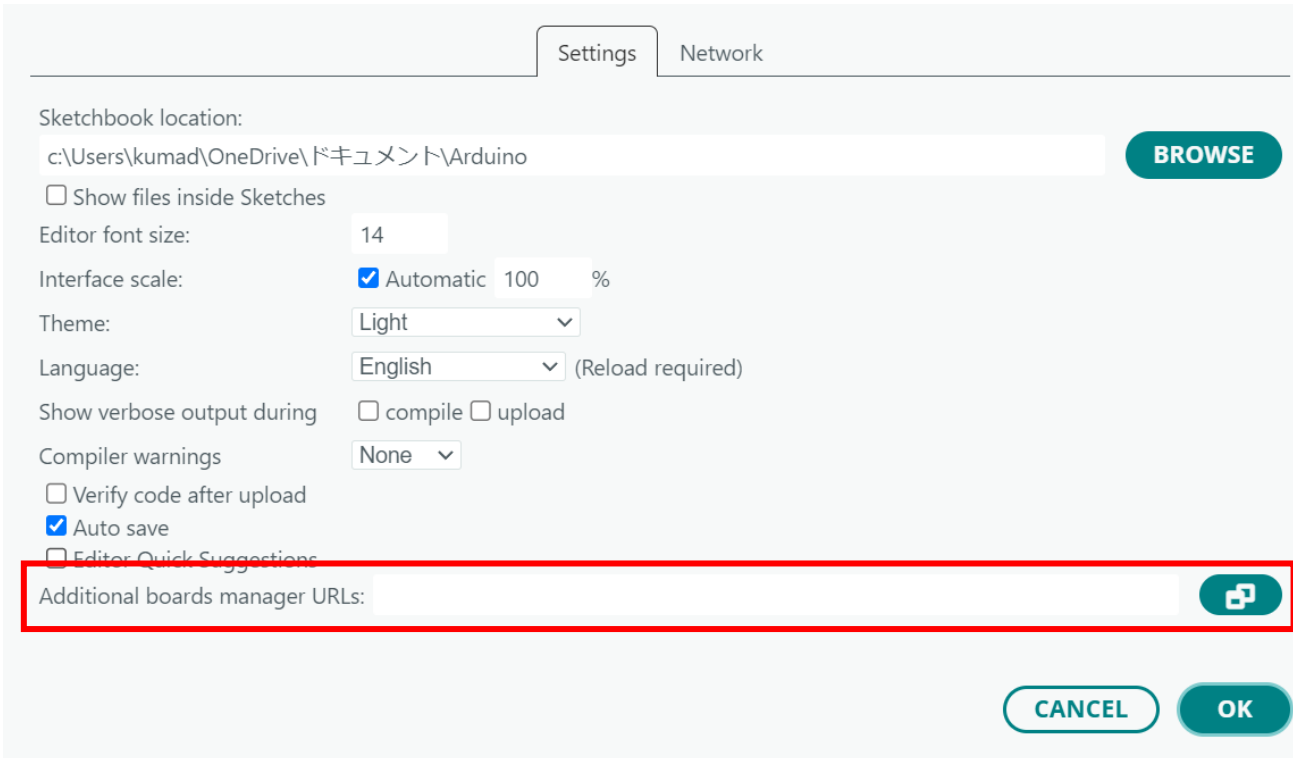
[Release notes](#)

- Download: <https://www.arduino.cc/en/software/>
- After installing the Arduino IDE, open robbit-esp.ino from the GitHub repository and configure the settings.

Program Upload: Board Settings

- Open `File` → `Preferences` → `Setting` → `Additional boards manager URLs` and enter the following URL.

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json



The screenshot shows the 'Settings' tab of the Arduino IDE. The 'Additional boards manager URLs' field at the bottom is highlighted with a red rectangle. Other settings visible include:

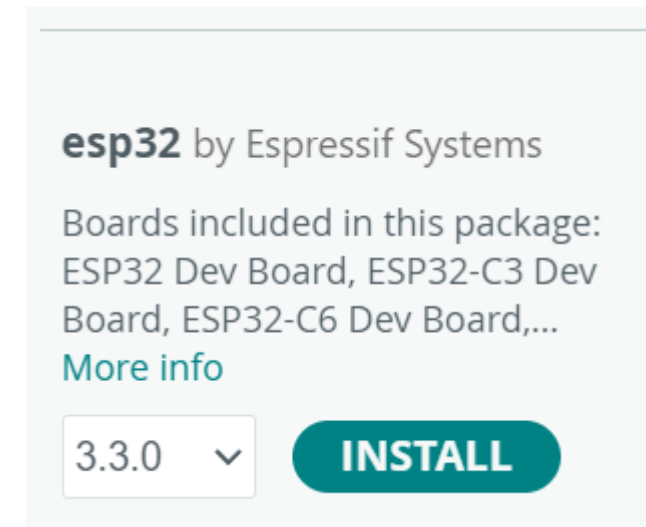
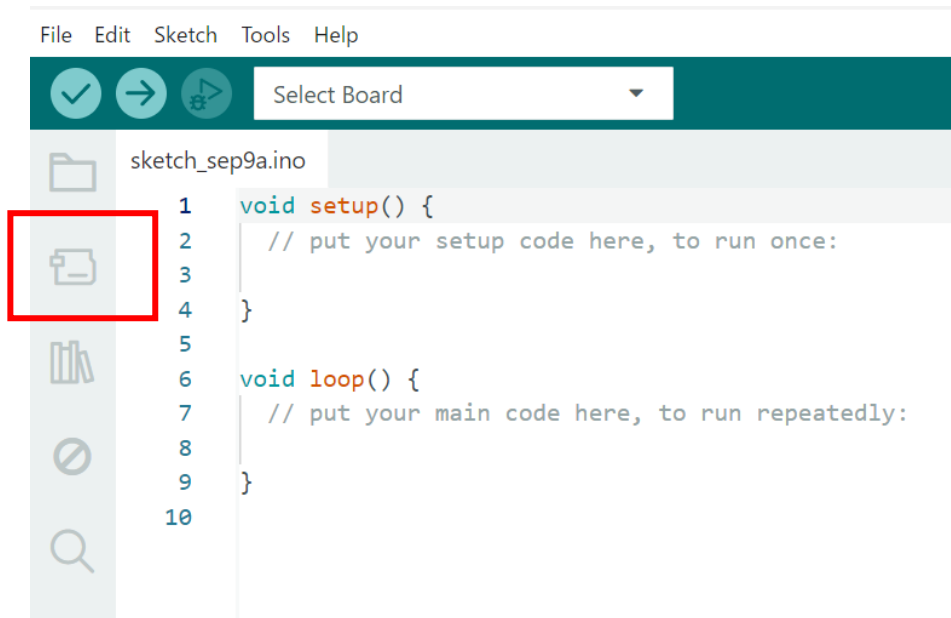
- Sketchbook location: c:\Users\kumad\OneDrive\ドキュメント\Arduino
- Editor font size: 14
- Interface scale: ☒ Automatic 100 %
- Theme: Light
- Language: English (Reload required)
- Show verbose output during: ☐ compile ☐ upload
- Compiler warnings: None
- ☐ Verify code after upload
- ☒ Auto save
- ☐ Editor Quick Suggestions

URL Entry Field

Program Upload: Board Settings

- Open `Tools` → `Board` → `Board Manager` and install `esp32` by Espressif Systems`.
- Select `Tools` → `Board` → `esp32` → `XIAO\_ESP32C3`.
- Under `Tools` → `Port`, select the port for the ESP32-C3.

The Board Manager can also be opened here



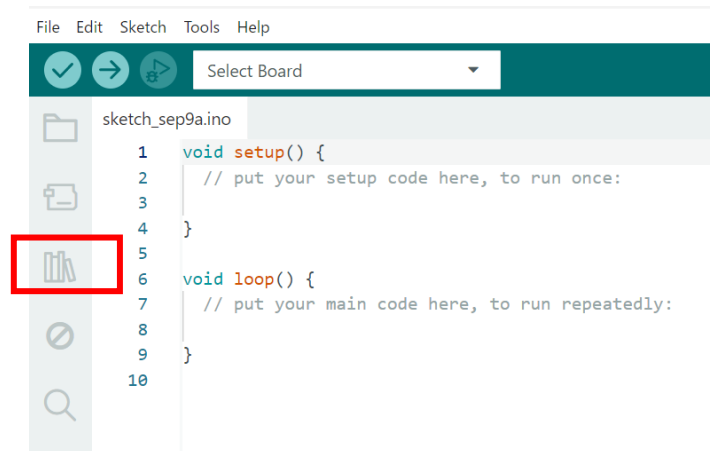
Board Package to Install

Program Upload: Library Setup

- Open `Tools` → `Manage libraries` and install the following libraries.
 - MPU6050 by Electric Cats
 - Madgwick by Arduino
 - BluetoothSerial by Henry Abrahamsen
- Add the following function to MadgwickAHRS.h in the Madgwick filter library.

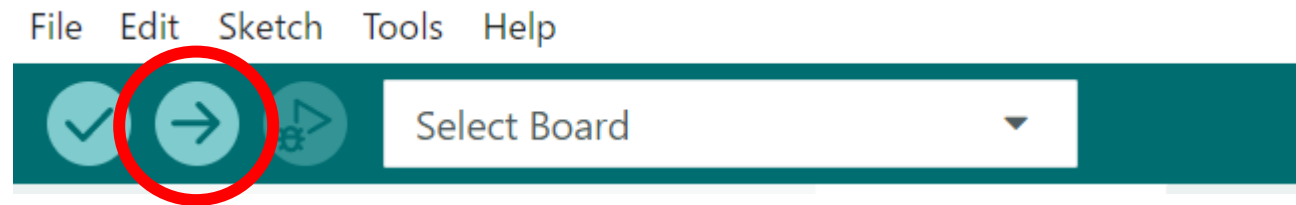
```
void setGain(float gain) { beta = gain; }
```

The Library Manager can
also be opened here



Program Upload: Upload

- Before uploading, verify the following:
 - The ESP32-C3 is connected to the PC.
 - The ESP32-C3 port is selected under `Tools` → `Port`.
 - `Tools` → `Board` → `esp32` → `XIAO\_ESP32C3` is selected.
- After verifying these settings, click Upload in the upper-left corner. The IDE compiles and uploads the program.



Upload Button

Operation Check

- After uploading, disconnect the ESP32-C3 from the PC, turn on robbit-esp, and verify its operation.
- Perform the operation check on a surface with some friction, such as carpet.
- Balancing in place is more difficult on a low-friction surface.

Operation Checklist

- When robbit-esp is tilted by hand, do the wheels rotate in the direction of the tilt?
- Do the wheels stop after the robot is tilted far from vertical? (The program stops them at 40° .)

Parameter Tuning: Environment Setup

- robbit-esp supports parameter tuning over BLE from a PC or smartphone.
- Using a PC
 - The PC workflow uses the Python bleak library, so create a Python virtual environment.
 - After creating the virtual environment, install bleak with `pip install bleak`.→ Go to page 22
- Using a Smartphone
 - Install a BLE communication app. → Go to page 25



BLE Scanner (available for Android and iOS)

Parameter Tuning: BLE Connection (PC)

- Turn on robbit-esp, wait briefly, and then run `ble_connect.py`.
- The script connects the PC to robbit-esp wirelessly. A successful connection produces the output below.
- The output shows the ESP32-C3 service UUID (``Service_uuid``), MAC address (``MAC_adress``), and characteristic UUID.
- The service and characteristic UUIDs should match the values defined near the beginning of the `.ino` file.

```
(venv) PS C:\Users\kumagai\ble-bcar> python .\ble_connect.py
connected 64:E8:33:89:F6:02
Service_uuid: 00001800-0000-1000-8000-00805f9b34fb
Service_uuid: 00001801-0000-1000-8000-00805f9b34fb
Service_uuid: 4fafc201-1fb5-459e-8fcc-c5c9c331914b
MAC_adress : 64:E8:33:89:F6:02
Characteristic_uuid : beb5483e-36e1-4688-b7f5-ea07361b26a8
input ("parameter number" "value") -> 
```

Parameter Tuning: Change Values (PC)

- At the `input ("parameter number" "value") ->` prompt, enter a parameter number and value in the format below. See page 28 for parameter numbers.

"parameter number" _ "value"

Here, _ represents a single ASCII space.

- The example below changes the V_MAX parameter to 200.

The result is displayed after the change is applied

```
input ("parameter number" "value") -> 6 200  
write V_MAX = 200
```

Parameter Tuning: Other Commands

- Enter `r` at the prompt to read the current parameter values.

```
input ("parameter number" "value") -> r
*-----*
target : -12.10,
Kp : 900.00,
Ki : 3000.00,
Kd : 20.00,
pwm_base : 40.00,
V_MAX : 200.00
*-----*
```

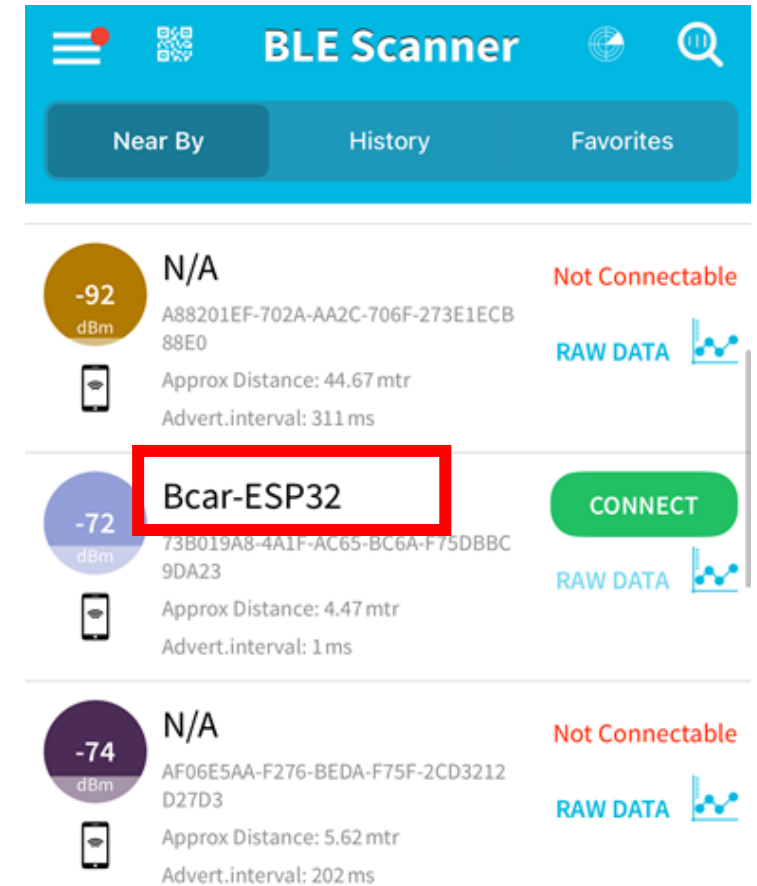
- Enter `e` at the prompt to disconnect from robbit-esp.

Parameter Tuning: BLE Connection (Smartphone)

- After turning on robbit-esp, the ESP32-C3 appears in BLE Scanner.
- The advertised device name is set in the following part of the .ino file.
- When operating multiple robbit-esp units, assign a different device name to each one.

Device-Name Setting

```
BLEDevice::init("Bcar-ESP32");  
BLEServer *pServer = BLEDevice::createServer();  
BLEService *pService = pServer->createService(SERVICE_UUID);
```



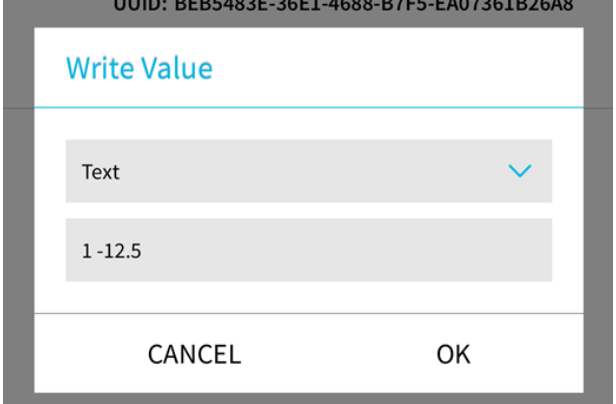
App Screen

Parameter Tuning: Change Values (Smartphone)

- Enter a parameter number and value in the following format.
See page 28 for parameter numbers.

"parameter number" _ "value"

Here, _ represents a single ASCII space.



UUID: BEB5483E-36E1-4688-B7F5-EA07361B26A8

Write Value

Text

1-12.5

CANCEL OK

Input Example

Parameter Tuning: Other Commands

- Use the app's read operation to retrieve all parameter values.
- The values are returned as the following comma-separated list.

TARGET, P_GAIN, I_GAIN, D_GAIN, PWM_Base, V_MAX

< Bcar-ESP32 DISCONNECT

Status CONNECTED

BONDED

CUSTOM SERVICE

^ 4FAFC201-1FB5-459E-8FCC-C5C9C331914B

PRIMARY SERVICE

CUSTOM CHARACTERISTIC R W

UUID: BEB5483E-36E1-4688-B7F5-EA07361B26A8

Properties: READ, WRITE

Value: -12.00, 900.00, 3000.00, 38.00, 40.00, 118.00

0x2D31322E30302C203930302E30302C20333030

Hex: 302E30302C2033382E30302C2034302E30302C2031382E30300A

Write Value: WRITE REQUEST



Parameter
Values

Parameter Tuning: Parameter Numbers

- Each tunable parameter is assigned a parameter number.

Parameter	Parameter Number	Description
Target	1	Target body angle
P_gain	2	Proportional gain
I_gain	3	Integral gain
D_gain	4	Derivative gain
PWM_Base	5	PWM signal offset
V_max	6	Maximum PWM signal value

- Loop Frequency

The loop frequency is the execution frequency of the main loop in `robbit-esp.ino`.

`D\_LOOP\_HZ` in `robbit-esp.ino` specifies the loop frequency.

Increasing the value makes the loop execute more frequently.

Specify the value in Hz.

```
#define D_LOOP_HZ    500
```

Example: 500 Hz

- Integral-Term Limit

Set an integral-term limit to implement anti-windup.

The value is set by `D\_I\_MAX` in `robbit-esp.ino`.

```
#define D_I_MAX      0.2 // 0.2,
```

Example: limit set to 0.2

Suggested Challenges

**Choose a performance goal for parameter tuning.
Examples are shown below.**

■ Difficulty: Easy

Balance for at least 10 minutes on a high-friction surface such as carpet.

■ Difficulty: Hard

**Balance for as long as possible on a low-friction surface.
Add disturbances such as wind or an incline.**